

**МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ**

**ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные базы
данных»**

**Тема: Простые отметки на карте - грибные места,
координаты участков, отметки на карте, маршруты.**

Студенты гр. 3343

Ермолаева В. А.

Иванов П. Д.

Синицкая Д. В.

Преподаватель

Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студенты

Ермолаева В. А.

Иванов П. Д.

Синицкая Д. В.

Группа 3343

Тема проекта: Простые отметки на карте - грибные места, координаты участков, отметки на карте, маршруты.

Исходные данные:

Необходимо реализовать веб-приложение, в котором можно делать простые заметки на карте. Отмечать интересные места, зоны, маршруты и тд. Искать и анализировать их, соотносить с объектами на карте.

Содержание пояснительной записки:

«Содержание»

«Введение»

«Качественные требования к решению»

«Сценарий использования»

«Модель данных»

«Разработка приложения»

«Вывод»

«Приложение»

Предполагаемый объем пояснительной записки:

Не менее 41 страницы.

Дата выдачи задания: 13.02.2026

Дата сдачи реферата: 20.05.2026

Дата защиты реферата: 20.05.2026

Студенты гр. 3343

Ермолаева В. А.

Иванов П. Д.

Синицкая Д. В.

Преподаватель

Заславский М.М.

АННОТАЦИЯ

Проект «Mushrooms Place» представляет собой веб-приложение для работы с персональными картами грибных мест. Система предоставляет возможность создавать карты, добавлять на них отметки различных типов (точки, маршруты и области), выполнять поиск и фильтрацию данных, а также анализировать собранную информацию с использованием статистических графиков.

В рамках индивидуального домашнего задания был проведён сравнительный анализ двух подходов к хранению данных: документно-ориентированного (MongoDB) и реляционного (PostgreSQL). Для обеих моделей выполнена оценка удельного объёма памяти, уровня избыточности данных и количества запросов, необходимых для реализации основных сценариев использования.

Результаты анализа показали, что для рассматриваемой предметной области более подходящей является документная модель хранения данных. Она обеспечивает меньшую избыточность, сокращает количество запросов при работе с иерархическими геоданными и позволяет использовать более гибкую схему данных.

В проекте используются следующие технологии: React, NestJS, MongoDB, Docker, Nginx и др.

Исходный код проекта и дополнительная информация доступны в репозитории: <https://github.com/moevm/nsql1h26-place>.

ANNOTATION

The «Mushrooms Place» project is a web application for working with personal maps of mushroom places. The system provides the ability to create maps, add various types of markers (points, routes, and areas), search and filter data, and analyze the collected information using statistical graphs.

As part of an individual homework assignment, a comparative analysis of two approaches to data storage was conducted: document-oriented (MongoDB) and relational (PostgreSQL). For both models, the specific amount of memory, the level of data redundancy, and the number of queries required to implement the main usage scenarios were estimated.

The results of the analysis showed that the document data storage model is more suitable for the subject area under consideration. It provides less redundancy, reduces the number of queries when working with hierarchical geodata, and allows for a more flexible data schema.

The application uses these instruments: React, NestJS, MongoDB, Docker, Nginx etc.

The project's source code and additional information are available in the repository: <https://github.com/moevm/nsql1h26-place>.

Оглавление

1.	Введение.....	6
2.	Сценарии использования.....	8
3.	Модель данных.....	14
4.	Разработанное приложение.....	29
5.	Вывод.....	36
6.	Приложения.....	38
7.	Используемая литература.....	41

1. Введение

Целью работы является создание веб-приложения «Mushrooms Place», предназначенного для ведения заметок на карте: грибных мест, участков леса, маршрутов. Приложение ориентировано на грибников, туристов и любителей природы. В ходе выполнения ИДЗ необходимо спроектировать модель данных, обосновать выбор СУБД и реализовать прототип с базовым набором функций.

а. Актуальность решаемой проблемы

Любители активного отдыха на природе часто фиксируют координаты найденных грибных мест, стоянок или интересных троп. Существующие сервисы либо избыточно сложны, либо не позволяют гибко управлять собственной коллекцией карт и объектов на них. Требуется лёгкое персонализированное решение, в котором пользователь может самостоятельно создавать тематические карты, добавлять точки, маршруты, области, обмениваться данными и получать простую аналитику по своим находкам.

б. Постановка задачи

Разработать веб-приложение для создания и ведения персональных карт грибных мест. Приложение должно обеспечивать регистрацию и аутентификацию пользователей, а также предоставлять возможность создания, редактирования и удаления карт с указанием страны, региона и уровня видимости. Пользователь может наносить на карту различные типы объектов: точки, обозначающие грибные места, области сбора и маршруты передвижения. Система должна поддерживать импорт данных карт из файлов формата JSON, экспорт данных в машиночитаемом формате, поиск по названиям карт и объектов, а также отображение статистики. Для хранения данных предполагается использование MongoDB.

в. Предлагаемое решение

Клиентская часть приложения реализована с использованием React и TypeScript. Для отображения интерактивной карты используется библиотека Leaflet, а управление состоянием приложения осуществляется с помощью Zustand. Серверная часть разработана на NestJS и предоставляет REST API с документацией Swagger. В качестве системы хранения данных используется MongoDB с поддержкой геопространственных индексов. Развёртывание приложения выполняется с использованием Docker Compose и прокси-сервера Nginx.

г. Качественные требования к решению

Работа с картой должна быть простой и понятной: пользователь должен иметь возможность добавлять объекты за один-два клика. Приложение должно корректно отображать несколько десятков объектов на карте. Система должна оставаться гибкой и позволять добавлять новые типы отметок и тегов без изменения структуры базы данных.

Безопасность обеспечивается разделением пользовательских данных, хранением паролей в зашифрованном виде. Архитектура приложения должна обеспечивать удобство расширения и сопровождения системы, в том числе возможность замены отдельных клиентских компонентов. Также предусматривается возможность масштабирования базы данных MongoDB, включая горизонтальное масштабирование при увеличении нагрузки.

2. Сценарии использования

Макеты UI

1. Страница входа/регистрации (рис. 1).

The image shows two side-by-side UI mockups for the login and registration pages of 'Mushroom Place'. Both pages have a light green background and a yellow header with the 'Mushroom Place' logo. The left mockup is for the login page, featuring a yellow box with the title 'Вход' (Login). It contains two input fields: 'Имя пользователя:' (Username) and 'Пароль:' (Password), both with placeholder text 'Введите имя пользователя' and 'Введите пароль' respectively. Below the fields are two buttons: a green 'Войти' (Login) button and a yellow 'Зарегистрироваться' (Register) button. The right mockup is for the registration page, featuring a yellow box with the title 'Регистрация' (Registration). It contains two input fields: 'Имя пользователя:' (Username) and 'Пароль:' (Password), both with placeholder text 'Введите имя пользователя' and 'Введите пароль' respectively. Below the fields are two buttons: a green 'Зарегистрироваться' (Register) button and a yellow 'Войти' (Login) button.

Рисунок 1 – страница входа/регистрации

2. Главный экран (рис. 2).

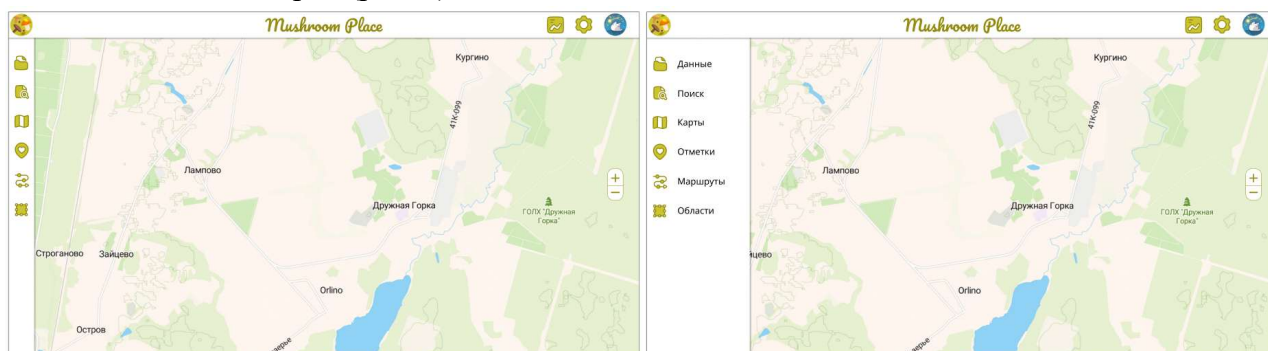


Рисунок 2 – главный экран

3. Функция импорта/экспорта (рис. 3).

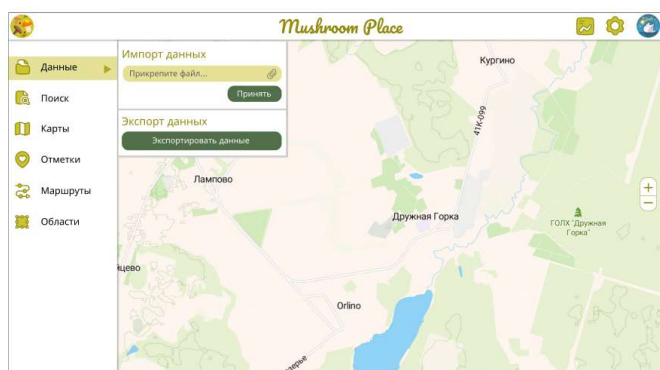


Рисунок 3 – функция импорта/экспорта

4. Функция поиска (рис. 4).

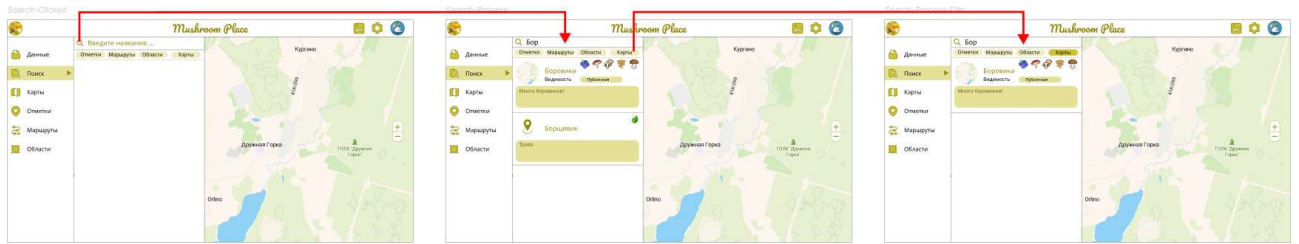


Рисунок 4 – функция поиска

5. Панель взаимодействия с картами (рис. 5).



Рисунок 5 – панель взаимодействия с картами

6. Панель взаимодействия с отметками на карте (рис. 6).

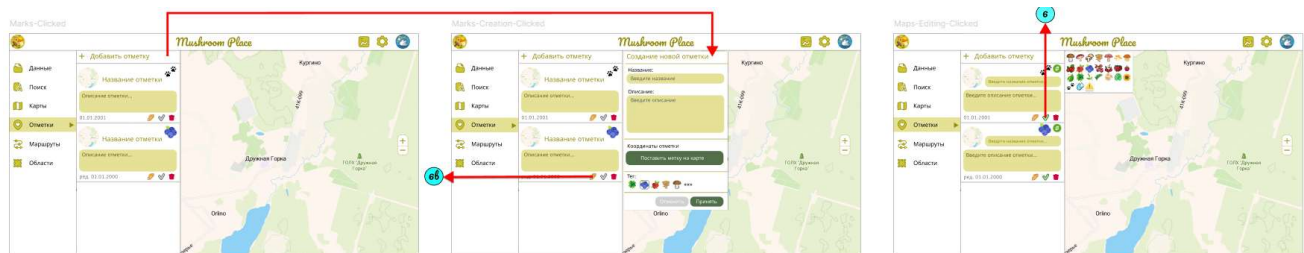


Рисунок 6 – панель взаимодействия с отметками на карте

7. Панель взаимодействия с маршрутами на карте (рис. 7).

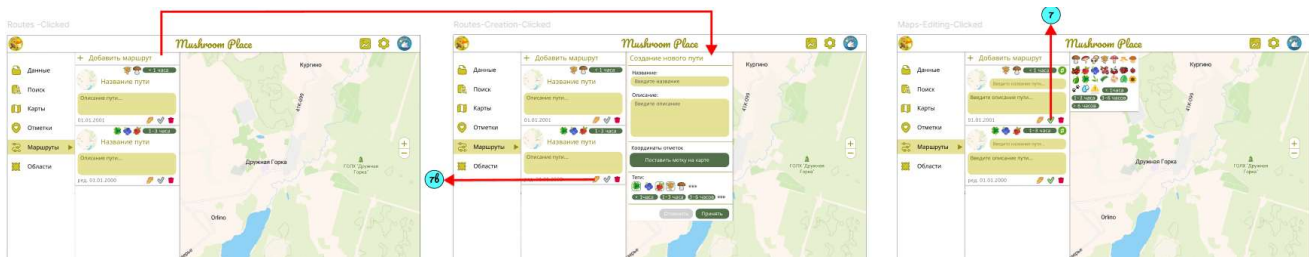


Рисунок 7 – панель взаимодействия с маршрутами на карте

8. Панель взаимодействия с областями на карте (рис. 8).

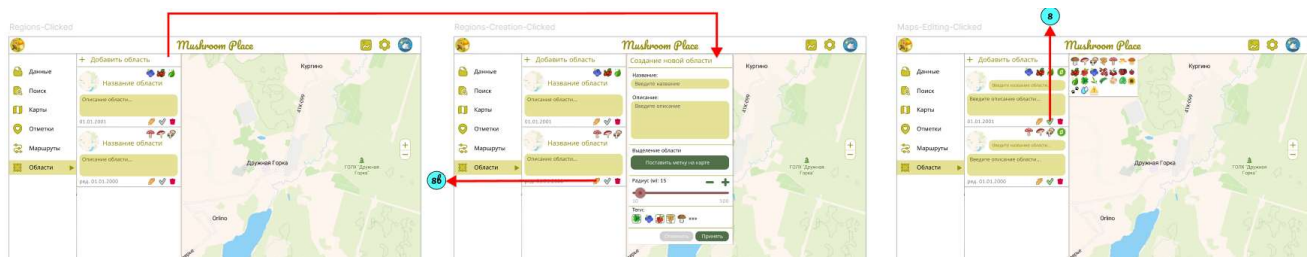


Рисунок 8 – панель взаимодействия с областями на карте

9. Экран настроек пользователя (рис. 9).

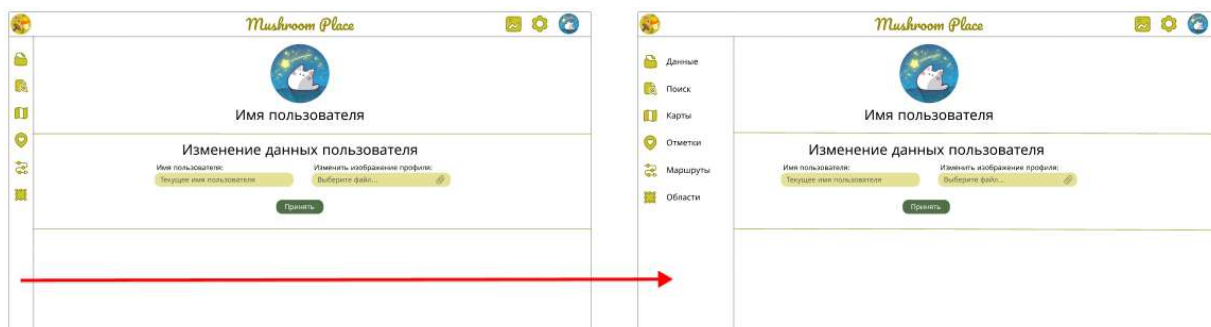


Рисунок 9 – настройки пользователя

10. Экран взаимодействия со статистикой (рис. 10).



Рисунок 10 – панель взаимодействия с картами

Сценарии использования для задач

1. Импорта данных.

А . Импорт данных (массовая загрузка из файла)

Предусловие: пользователь авторизован и находится на главном экране приложения.

Основной сценарий:

1. Пользователь нажимает кнопку «Данные» в боковом меню.
2. В открывшемся разделе находит блок «Импорт данных» и нажимает кнопку «Прикрепите файл формата .json...».
3. Проводник устройства предлагает выбрать файл. Пользователь выбирает подготовленный файл формата .json.
4. Пользователь подтверждает импорт нажатием кнопки «Открыть».
5. Система проверяет расширение и структуру файла.
6. При успешной проверке сервер создает новые карты и наполняет их объектами.

Альтернативный сценарий:

1. Файл имеет неподдерживаемый формат – пользователь видит сообщение «Нет

- элементов, удовлетворяющих условиям поиска». Импорт не выполняется.
2. Файл повреждён или содержит некорректные данные – система выводит уведомление «Ошибка при импорте данных».
 3. Пользователь закрывает диалог выбора файла (нажимает кнопку «Отмена») – импорт прекращается, никаких изменений в списке карт не происходит.

Результат: данные успешно добавлены в систему и доступны пользователю для просмотра, редактирования и анализа.

В. Ручной ввод данных (создание карт и объектов вручную)

Предусловие: пользователь авторизован и находится на главном экране приложения.

Основной сценарий (создание карты):

1. Пользователь нажимает кнопку «Карты» в боковом меню, нажимает «Добавить карту».
2. Заполняет поля формы: название карты (обязательно), описание (опционально), область (обязательно, для географической привязки), переключатель видимости (публичная/приватная).
3. Нажимает кнопку «Принять».
4. Новая карта сохраняется и отображается в списке.

Ручное добавление объектов (точка, область, маршрут):

1. Пользователь убеждается, что в боковом меню выбрана нужная карта.
2. Для добавления отметки: переходит во вкладку «Отметки», нажимает «Добавить отметку». Вводит название, описание, указывает кликом на карте координаты, при необходимости выбирает теги. Нажимает «Принять».
3. Для добавления области: переходит во вкладку «Области», нажимает «Добавить область». Вводит название, описание, кликом задаёт центр области и радиус (в метрах), при необходимости выбирает теги. Нажимает «Принять».
4. Для добавления маршрута: переходит во вкладку «Маршруты», нажимает «Добавить маршрут». Вводит название, описание, последовательными кликами на карте ставит точки пути (минимум две), при необходимости выбирает теги. Нажимает «Принять».
5. Система сохраняет объект, и он отображается на карте в соответствующем стиле (точка – маркер, область – полупрозрачный круг, маршрут – линия).

Альтернативный сценарий:

1. Не заполнены обязательные поля – система выводит предупреждение.
2. Для маршрута указана только одна точка – выводится сообщение «Добавьте минимум две отметки маршрута».
3. Пользователь нажимает «Отмена» или закрывает форму – все введенные данные теряются, объект/карта не создается.

Результат: новая карта или объект (отметка, область, маршрут) успешно созданы и визуализированы на карте.

2. Представления данных.

Отображение объектов на карте: после выбора карты в боковой панели все принадлежащие ей объекты автоматически подгружаются через API. Точки отображаются маркерами. Области визуализируются как окружности с центром и радиусом, заданными пользователем. Маршруты рисуются в виде линий с опорными точками.

Табличное представление: в панели слева можно переключиться в режим списка, где объекты представлены в виде карточек с полями: название, описание и дата создания.

Поиск данных: пользователь вводит поисковый запрос в строку в боковой панели. Есть несколько вариантов поиска: поиск по названию, поиск по дате, поиск по тегам, поиск по описанию. Можно применить фильтры для сужения области поиска: отметки, маршруты, области, карты. Система выполняет поиск и возвращает список совпадений.

Альтернативные варианты: если поисковый запрос не дал результатов – отображается сообщение «Ничего не найдено».

3. Анализа данных.

Предусловие: пользователь авторизован и имеет хотя бы одну карту или объекты.

Основной сценарий:

1. Пользователь переходит в раздел «Статистика».
2. Пользователь выбирает значения для осей икс и игрек, при необходимости настраивает фильтры, категории и теги.
3. Отображается сформированная статистика.

Альтернативный сценарий:

1. Не выбран один из обязательных параметров (ось X и Y) – график не строится.
2. У пользователя нет данных для выбранной комбинации фильтров – вместо графика отображается сообщение «Нет данных для выбранных фильтров».

Результат: на экране отображается интерактивный график, позволяющий быстро оценить распределение объектов по типам.

4. Экспорта данных.

Предусловие: пользователь авторизован и имеет хотя бы одну созданную/импортированную карту.

Основной сценарий:

1. Пользователь нажимает кнопку «Данные» в боковом меню.
2. Нажимает кнопку «Экспорт данных».
3. Сервер извлекает все объекты, формирует файл в формате .json и отправляет его клиенту.
4. Клиент нажимает на созданный файл, отображенный на панели экспорта.
5. Браузер начинает загрузку файла на устройство пользователя.

Альтернативный сценарий:

1. Ошибка соединения – появляется сообщение «Экспорт не удался», загрузка не начинается.

Результат: файл с копией всех карт и объектов сохранён на локальном устройстве в формате .json, готовый для импорта в другие приложения или анализа в сторонних инструментах.

Вывод о преобладающих операциях

В системе доминируют операции чтения: просмотр карт и объектов на карте, поиск, получение статистики. Запись происходит значительно реже (создание или импорт карты, добавление нескольких объектов за сеанс). Это оправдывает выбор документной модели, ориентированной на быстрые выборки по индексам.

3. Модель данных

Нереляционная модель данных MONGODB

1. Графическое представление

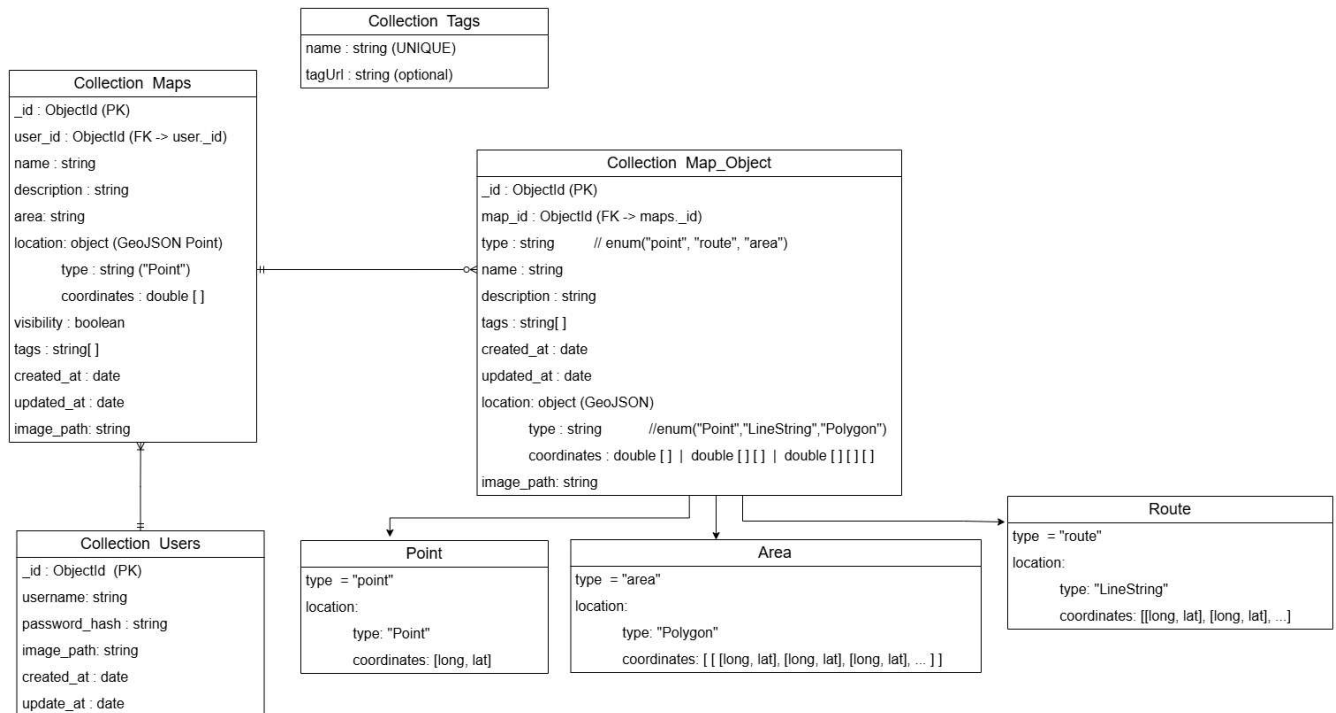


Рисунок 11 – нереляционная модель БД

2. Описание назначений коллекций, типов данных и сущностей

Коллекции:

1. Users – пользователи. Коллекция хранит данные пользователей системы. Каждый документ соответствует одному пользователю. Содержит поля:

- `_id` – уникальный идентификатор пользователя, является первичным ключом
- `username` – логин пользователя
- `password_hash` – хеш пароля
- `image_path` – ссылка на аватар пользователя
- `created_at` – дата создания
- `updated_at` – дата редактирования, по умолчанию NULL

2. Tags – теги. Коллекция содержит теги, используемые для категоризации маршрутов, отметок и карт. Каждый документ описывает один тег. Содержит поля:

- `name` – уникальное название тега

- `tagUrl` (опционально) – ссылка на изображение тега

3. `Maps` – карты. Коллекция хранит карты, созданные пользователем. Каждый документ описывает одну карту. Содержит поля:

- `_id` – уникальный идентификатор карты, первичный ключ
- `user_id` – ссылка на пользователя. Соответствует полю `_id` в коллекции `Users`
- `name` – название карты
- `description` – описание карты
- `country` - страна
- `area` – область карты
- `location` – геопозиция центра карты (GeoJSON Point), вложенный документ
- `type` – тип, строка
- `coordinates` – координаты, массив из двух чисел `double`
- `visibility` – видимость карты (публичная/приватная)
- `tags` – массив строк (названия тегов)
- `created_at` – дата создания
- `updated_at` – дата редактирования, по умолчанию `NULL`
- `image_path` – ссылка на аватар карты

4. `Map_Object` – объекты, которые можно отмечать на карте (отметка, маршрут, область). Каждый документ описывает один объект. Содержит поля:

- `_id` – уникальный идентификатор объекта, первичный ключ
- `map_id` – ссылка на карту, к которой принадлежит объект. Соответствует полю `_id` в коллекции `Maps`
- `type` – тип объекта ("point", "area", "route")
- `name` – название объекта
- `description` – описание объекта
- `tags` – теги, массив строк

- `created_at` – дата создания
- `updated_at` – дата редактирования, по умолчанию NULL
- `location` – геопозиция объекта (GeoJSON), вложенный документ
- `type` – тип, строка
- `coordinates` – координаты, массив чисел double
- `image_path` – ссылка на аватар отметки

3. Оценка удельного объема информации, хранимой в модели

Общий объем памяти, необходимый для хранения всех данных, зависит от количества карт N (документы коллекции `Maps`), так как она является центральной. Остальные коллекции связаны с картой через средние статистические коэффициенты, полученные на основе типичного использования:

- Количество пользователей: $N/5$ (в среднем один пользователь создаёт 5 карт)
- Количество отметок: $100N$ (100 отметок на карту)
- Количество областей: $5N$ (5 областей на карту)
- Количество маршрутов: $10N$ (10 маршрутов на карту)
- Количество тегов: $30N$ (30 тегов на карту)
- Среднее количество точек маршрута (waypoints) на один маршрут (входят в состав документов коллекции `Routes`): $200N$ (на маршруте 20 точек)
- Среднее количество тегов на карту: 3, на область: 3, на маршрут: 3, на отметку: 1

Накладные расходы, учтённые при расчёте объёма БД (в байтах):

- BSON overhead (структура документа MongoDB). Для документа с 10–15 полями служебная информация ~10–50 байт к размеру каждого документа.
- Накладные расходы на массивы и вложенные документы. Массивы строк (tags) или массивы чисел (координаты) хранятся как вложенные документы. В среднем для массива из 3 строковых тегов добавляем ~30–50 байт, для `LineString` с 20 точками – ~20–40 байт, для `Polygon` с 50 точками – ~50–100 байт.

- Накладные расходы на MVCC. В оценке предполагаем, что ~1–2 версии документа хранятся временно, добавляя ~10–20% к объёму данных.
- TOAST (хранение полей переменной длины). В MongoDB большие строки хранятся inline до ~16 МБ, но для документа с 500–1000 байт это добавляет накладные расходы на заголовки и указатели → ~10–15%.
- WAL (журналы записи). В оценке учтено усреднённое увеличение объёма данных на 5–10%.
- Индексы и bitmap для NULL. Первичные ключи, частичные индексы и другие часто используемые поля (_id, user_id, map_id) требуют дополнительного места. ObjectId индекс → ~12–16 байт на документ. Строковый индекс (например, username) → 1–3 байта на символ + структура индекса B-tree → добавлено условно ~5–10% к объёму. Bitmap для хранения NULL значений полей → ~1–2 байт на поле.
- Суммарно, для оценки порядка величины, добавляем ~15% к «чистому» объёму документа.

Размеры документов в коллекциях (в байтах):

- Users – чистый объём 1100–1400, накладные расходы ~15%, расчётный объём ~1800–1850
- Maps – чистый объём 300–1000, накладные расходы ~15%, расчётный объём ~1077
- Map_Object – чистый объём 3360–3860, накладные расходы ~15%, расчётный объём ~4196
- Tags – чистый объём 700–2000, накладные расходы ~15%, расчётный объём ~1375 (среднее по распределению: 60% Point, 30% LineString, 10% Polygon)

Суммарный объём вычисляется как сумма произведений количества документов на их размер. После подстановки всех выражений и упрощения получаем линейную зависимость от N:

$$V(N) = 1.15 * (368.4N + 4196N + 158125N + 32310N) \text{ байт} \sim 0.223N \text{ Мбайт}$$

4. Запросы к модели, с помощью которых реализуются сценарии использования

1. Сценарий: создание новой карты пользователем

```
db.maps.insertOne({ user_id: ObjectId("507f1f77bcf86cd799439011"), name: "Грибные места", description: "Очень много грибов", country: "Россия", area: "Тамбовская область", location: { type: "Point", coordinates: [56.95, 24.10] }, visibility: true, tags: ["Лисичка"], created_at: new Date(), updated_at: null, image_path: "/maps/tombov.png" })
```

Для выполнения сценария требуется 1 запрос. Задействованные коллекции: 1 (Maps).

2. Сценарий: добавление грибной отметки на карту

```
db.map_object.insertOne({ map_id: ObjectId("507f1f77bcf86cd799439012"), type: "point", name: "Белые грибы", description: "Найдено много белых грибов", tags: ["Лисичка"], created_at: new Date(), updated_at: null, location: { type: "Point", coordinates: [56.951, 24.105] }, image_path: "/points/mushroom.png" })
```

Для выполнения сценария требуется 1 запрос. Задействованные коллекции: 1 (Map_Object).

3. Сценарий: построение маршрута к грибному месту

```
db.map_object.insertOne({ map_id: ObjectId("507f1f77bcf86cd799439012"), type: "route", name: "Маршрут к грибной поляне", description: "Короткий путь через лес", tags: [], created_at: new Date(), updated_at: null, location: { type: "LineString", coordinates: [ [56.950, 24.100], [56.951, 24.102], [56.952, 24.105] ] }, image_path: "/routes/mushroom_route.png" })
```

Для выполнения сценария требуется 1 запрос. Задействованные коллекции: 1 (Map_Object).

4. Сценарий: поиск грибных отметок по тегу

```
db.map_object.find( { tags: "Лисичка", type: "point" }, { name: 1, description: 1, "location.coordinates": 1, type: 1 } )
```

Для выполнения сценария требуется 1 запрос. Задействованные коллекции: 1 (Map_Object).

5. Сценарий: поиск по названию среди всех карт, областей, отметок и маршрутов

Выполняется отдельный поиск по Maps и Map_Object (points, areas, routes), объединение на стороне приложения

```
let mapsResults = db.maps.find( { name: { $regex: "поисковый_запрос", $options: "i" } }, { name: 1, description: 1, "location.coordinates": 1 } ).map(doc => ({ object_type: "map", id: doc._id, name: doc.name, description: doc.description, lat: doc.location?.coordinates[0], long: doc.location?.coordinates[1] }));
```

```
let mapObjectsResults = db.map_object.find( { name: { $regex: "поисковый_запрос",
$options: "i" } }, { name: 1, description: 1, type: 1, "location.coordinates": 1 } ).map(doc =>
({ object_type: doc.type, id: doc._id, name: doc.name, description: doc.description, lat:
doc.location?.coordinates[0] || null, long: doc.location?.coordinates[1] || null }));
```

Для выполнения сценария требуется 2 запроса. Задействованные коллекции: 2 (Maps, Map_Object).

Аналог модели данных для SQL СУБД – PostgreSQL

1. Графическое представление

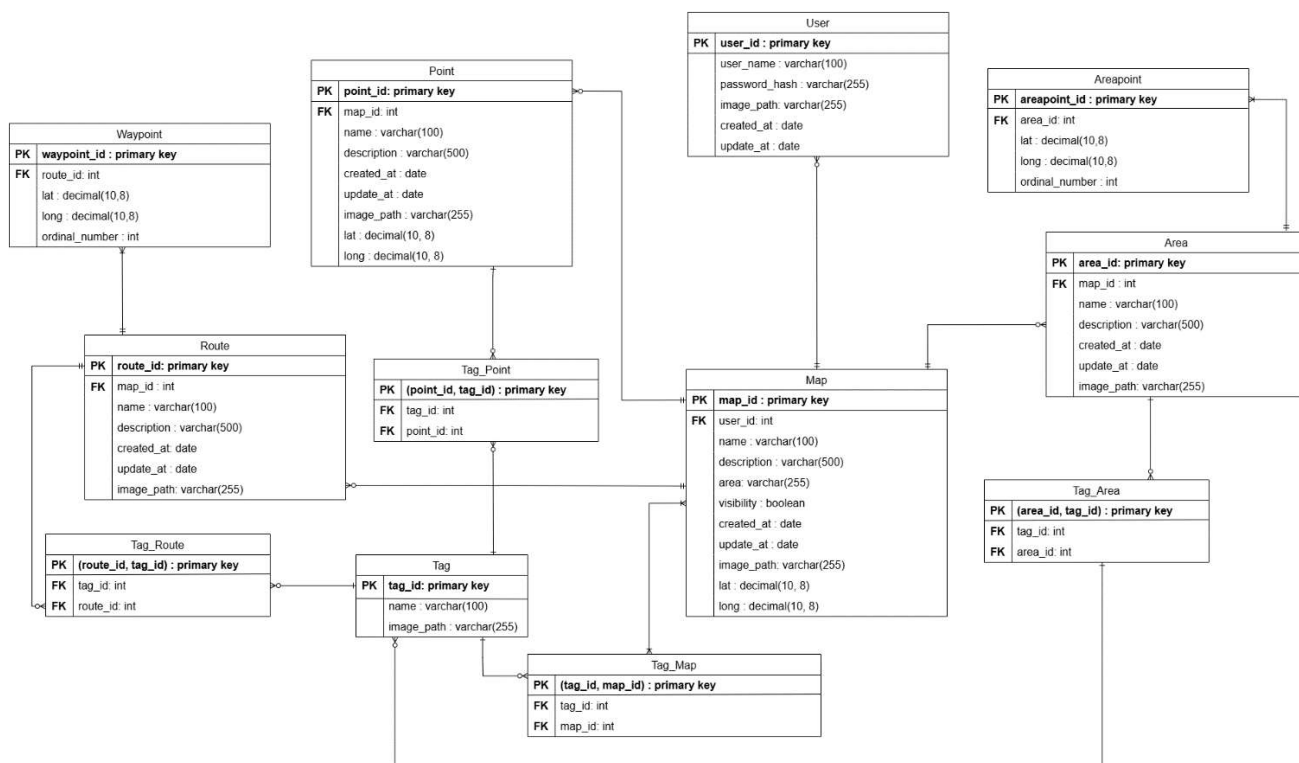


Рисунок 12 – реляционная модель БД

2. Описание назначений коллекций, типов данных и сущностей

Сущности:

1. User – пользователь, сущность хранит все данные пользователя. Содержит поля:

- user_id – уникальный идентификатор пользователя, является первичным ключом
- user_name – логин
- password_hash – хэш пароля
- image_path – ссылка на аватар пользователя

- created_at – дата создания
- update_at – дата редактирования, по умолчанию NULL

2. Map – карта, центральная сущность бд, хранит данные для создания карты местности, выбранной пользователем. Содержит поля:

- map_id – уникальный идентификатор карты, является первичным ключом
- user_id – ссылка на пользователя, внешний ключ
- name – название карты
- description – описание карты
- area – область карты
- visibility – видимость карты (публичная/приватная)
- create_at – дата создания
- update_at – дата редактирования, по умолчанию NULL
- image_path – ссылка на аватар карты
- lat – координата широты центра карты
- long – координата долготы центра карты

3. Point – отметка, сущность хранит данные отметок, поставленных пользователем на карте. Содержит поля:

- point_id – уникальный идентификатор отметки, является первичным ключом
- map_id – уникальный идентификатор карты, является внешним ключом
- name – название отметки
- description – описание отметки
- create_at – дата создания
- update_at – дата редактирования, по умолчанию NULL
- image_path – ссылка на аватар отметки
- lat – координата широты отметки
- long – координата долготы отметки

4. Area – область, сущность хранит данные областей, выделенных пользователем на карте. Содержит поля:

- area_id – уникальный идентификатор области, является первичным ключом
- map_id – уникальный идентификатор карты, является внешним ключом
- name – название области
- description – описание области
- create_at – дата создания
- update_at – дата редактирования, по умолчанию NULL
- image_path – ссылка на аватар области

5. Areapoint – точка границы, сущность хранит данные о точках, входящих в границу области. Содержит поля:

- areapoint_id – уникальный идентификатор точки маршрута, является первичным ключом
- area_id – уникальный идентификатор области, является внешним ключом
- lat – координата широты точки
- long – координата долготы точки
- ordinal_number – порядковый номер точки на границе области

6. Route – маршрут, сущность хранит данные маршрута, проложенного пользователем на карте. Содержит поля:

- route_id – уникальный идентификатор маршрута, является первичным ключом
- map_id – уникальный идентификатор карты, является внешним ключом
- name – название маршрута
- description – описание маршрута
- create_at – дата создания
- update_at – дата редактирования, по умолчанию NULL
- image_path – ссылка на аватар маршрута

7. Waypoint – точка маршрута, сущность хранит данные о точках, входящих в маршрут. Содержит поля:

- waypoint_id – уникальный идентификатор точки маршрута, является первичным ключом
- route_id – уникальный идентификатор маршрута, является внешним ключом
- lat – координата широты точки
- long – координата долготы точки
- ordinal_number – порядковый номер точки на маршруте

8. Tag – тег, сущность хранит данные о тегах, предоставляемых пользователем на картах, отметках, маршрутах и областях. Содержит поля:

- tag_id – уникальный идентификатор тега, является первичным ключом
- name – название тега
- image_path – ссылка на изображение тега

Связи сущностей:

1. Tag_Map – таблица связи сущностей Tag и Map, связывает тег с картой, на которой он был поставлен. Содержит поля:

- (tag_id, map_id) – уникальный составной идентификатор, является первичным ключом
- tag_id – уникальный идентификатор тега, является внешним ключом
- map_id – уникальный идентификатор карты, является внешним ключом

2. Tag_Area – таблица связи сущностей Tag и Area, связывает тег с областью, на которой он был поставлен. Содержит поля:

- (area_id, tag_id) – уникальный составной идентификатор, является первичным ключом
- tag_id – уникальный идентификатор тега, является внешним ключом
- area_id – уникальный идентификатор области, является внешним ключом

3. Tag_Route – таблица связи сущностей Tag и Route, связывает тег с маршрутом, на котором он был поставлен. Содержит поля:

- (route_id, tag_id) – уникальный составной идентификатор, является первичным ключом
- tag_id – уникальный идентификатор тега, является внешним ключом
- route_id – уникальный идентификатор маршрута, является внешним ключом

4. Tag_Point – таблица связи сущностей Tag и Point, связывает тег с отметкой.

Содержит поля:

- (point_id, tag_id) – уникальный составной идентификатор, является первичным ключом
- tag_id – уникальный идентификатор тега, является внешним ключом
- point_id – уникальный идентификатор отметки, является внешним ключом

3. Оценка удельного объема информации, хранимой в модели

Общий принцип оценки: в реляционной модели данные хранятся в таблицах фиксированной структуры. В отличие от документо-ориентированной модели, здесь отсутствует хранение имён полей внутри записей, а также вложенные структуры. Размер строковых полей оценивается исходя из максимальной длины и кодировки UTF-8 (до 3 байт на символ). Числовые поля имеют фиксированный размер. Как и в нереляционной модели, расчёты являются приближенными и отражают порядок величины, без учёта всех накладных расходов СУБД.

Общий объем памяти, необходимый для хранения всех данных, зависит от количества карт N (сущность Map), так как она является центральной. Остальные сущности связаны с картой через средние статистические коэффициенты, полученные на основе типичного использования:

- Количество пользователей: $N/5$ (в среднем один пользователь создаёт 5 карт)
- Количество отметок: $100N$ (100 отметок на карту)
- Количество областей: $5N$ (5 областей на карту)
- Количество маршрутов: $10N$ (10 маршрутов на карту)
- Количество точек маршрута: $200N$ (на маршруте 20 точек)
- Количество точек границ областей : $250N$ (на границу 50 точек)
- Количество тегов: $30N$ (30 тегов на карту)

- Количество связей "тег-карта": 3N (3 тега на карту)
- Количество связей "тег-область": 15N (3 тега на область)
- Количество связей "тег-маршрут": 30N (3 тега на маршрут)
- Количество связей "тег-отметка": 100N (1 тег на отметку)

Размеры записей (в байтах):

- User – ~1840
- Map – ~4120
- Point – ~2570
- Area – ~2570
- Route – ~2570
- Waypoint – ~28
- Areapoint – ~28
- Tag – ~1065
- Tag_Map, Tag_Area, Tag_Route, Tag_Point – каждая запись по ~8

Учёт накладных расходов: в реляционной СУБД дополнительно учитываются: индексы (PK, FK) → ~10–15%, выравнивание данных → ~5%, служебные структуры хранения → ~5%. Итого добавляем ~20% к объёму.

Суммарный объём вычисляется как сумма произведений количества записей на их размер. После подстановки всех выражений и упрощения получаем линейную зависимость от N:

$$V(N) = 1.2 * 185000N \text{ байт} \sim 0.22N \text{ Мбайт}$$

4. Запросы к модели, с помощью которых реализуются сценарии использования

1. Сценарий: создание новой карты пользователем

INSERT INTO "Map" (user_id, name, description, area, visibility, created_at, image_path, lat, long) VALUES (1, 'Грибные места', 'Очень много грибов', 'Тамбовская область', 1, '2026-03-06', '/maps/tambov.png', 56.95, 24.10);

Для выполнения сценария требуется 1 запрос. Количество задействованных таблиц: 1

таблица (Map).

2. Сценарий: добавление грибной отметки на карту

```
INSERT INTO "Point" (map_id, name, description, created_at, image_path, lat, long)
VALUES (1, 'Белые грибы', 'Найдено много белых грибов', '2026-03-06', '/points/mushroom.png',
56.951, 24.105);
```

Для выполнения сценария требуется 1 запрос. Количество задействованных таблиц: 1
таблица (Point).

3. Сценарий: построение маршрута к грибному месту

```
- создание маршрута INSERT INTO "Route" (map_id, name, description, created_at,
image_path) VALUES (1, 'Маршрут к грибной поляне', 'Короткий путь через лес', '2026-03-
06', '/routes/mushroom_route.png');
```

```
- добавление точек маршрута INSERT INTO "Waypoint" (route_id, lat, long,
ordinal_number) VALUES (1, 56.950, 24.100, 1), (1, 56.951, 24.102, 2), (1, 56.952, 24.105, 3);
```

Для выполнения сценария требуется 1 + W запросов, где W - количество точек маршрута. Количество задействованных таблиц: 2 (Route, Waypoint).

4. Сценарий: поиск грибных отметок по тегу

```
SELECT p.name, p.description, p.lat, p.long FROM "Point" p JOIN "Tag_Point" tp ON
p.point_id = tp.point_id JOIN "Tag" t ON tp.tag_id = t.tag_id WHERE t.name = 'Лисичка';
```

Для выполнения сценария требуется 1 запрос. Сложность поиска зависит от количества отметок (P) и количества связей в таблице Tag_Point (TP). При наличии индексов по point_id, tag_id и Tag.name сложность близка к $O(\log(P) + \log(TP))$. Количество задействованных таблиц: 3 таблицы (Point, Tag_Point, Tag).

5. Сценарий: поиск по названию среди всех карт, областей, отметок и маршрутов

```
SELECT 'map' AS object_type, map_id AS id, name, description, lat, long FROM "Map"
WHERE name LIKE '%поисковый_запрос%' UNION ALL SELECT 'area', area_id, name,
description, NULL, NULL FROM "Area" WHERE name LIKE '%поисковый_запрос%' UNION
ALL SELECT 'point', point_id, name, description, lat, long FROM "Point" WHERE name LIKE
'%поисковый_запрос%' UNION ALL SELECT 'route', route_id, name, description, NULL, NULL
FROM "Route" WHERE name LIKE '%поисковый_запрос%';
```

Для выполнения сценария требуется 1 запрос (UNION четырёх таблиц). Сложность поиска константа (зависит от наличия индекса). Количество задействованных таблиц: 4

таблицы (Map, Area, Point, Route).

Сравнение моделей

1. Удельный объем

Объём данных в обеих моделях зависит от количества карт N , поскольку карта является центральной сущностью системы. Все остальные объекты (пользователи, отметки, области, маршруты и теги) связаны с картами через средние статистические коэффициенты использования.

- Для нереляционной модели получена формула общего объёма данных: $V(N) \approx 0,223 \cdot N$ Мбайт
- Для реляционной модели: $V(N) \approx 0,22 \cdot N$ Мбайт

Объём хранения в моделях практически одинаковый. Реляционная модель занимает немного меньше памяти, однако разница незначительна. Причины различий:

- В нереляционной модели: используется ObjectId (12 байт), присутствуют накладные расходы BSON, хранение имён полей и структур документов, вложенные структуры (массивы координат, тегов).
- В реляционной модели: используются целочисленные ключи (4 байта), отсутствует хранение структуры записи, данные распределены по таблицам.

Обе модели демонстрируют линейный рост объёма данных, а различия по памяти находятся в пределах погрешности оценки.

2. Запросы по отдельным юзкейсам

Важным критерием эффективности модели является количество запросов, необходимых для выполнения типичных операций системы.

Сценарии использования:

1. Создание новой карты

- Нереляционная модель — 1 запрос
- Реляционная модель — 1 запрос
- Связь хранится через user_id (FK), поэтому дополнительная таблица не используется → модели эквивалентны

2. Добавление отметки на карту

- Нереляционная модель — 1 запрос
- Реляционная модель — 1 запрос
- Связь с картой и тегом хранится прямо в таблице Point → модели эквивалентны

3. Построение маршрута

- Нереляционная модель — 1 запрос
- Реляционная модель — $1 + W$ запросов
- MongoDB — вложенный массив координат, SQL — отдельные записи Waypoint → нереляционная модель значительно эффективнее

4. Поиск отметок по тегу

- Нереляционная модель — 1 запрос
- Реляционная модель — 1 запрос (JOIN)
- Формально эквивалентно по количеству операций, однако стоимость выполнения различается. При наличии индексов обе модели дают сопоставимую производительность, без индексов нереляционная модель может работать медленнее, реляционная модель более предсказуема за счёт оптимизированных JOIN.

5. Поиск по названию среди различных типов объектов

- Нереляционная модель — 2 запроса (Maps + Map_Object)
- Реляционная модель — 1 запрос (UNION)
- Реляционная модель компактнее по количеству запросов, однако фактическая стоимость выполнения сопоставима. По количеству запросов реляционная модель проще, по фактической стоимости выполнения обе модели близки, выбор зависит от архитектуры: SQL — централизованная обработка в СУБД, MongoDB — распределение нагрузки между БД и приложением.

3. Вывод

Для данного приложения – картографического сервиса с возможностью создания карт, постановки отметок, построения маршрутов и выделения областей – документо-

ориентированная модель (MongoDB) является более предпочтительной.

Обоснование:

- Основные сценарии — запись и изменение данных, где MongoDB работает эффективнее
- Маршруты и области имеют иерархическую структуру, что естественно ложится в JSON
- Количество запросов при работе с геоданными существенно меньше
- Объём данных сопоставим с SQL
- Избыточность ниже (~12% против ~20%)
- Модель проще масштабируется и расширяется

4. Разработанное приложение

Краткое описание

Разработанное приложение представляет собой распределённую клиент-серверную систему, состоящую из нескольких независимых сервисов, взаимодействующих между собой посредством HTTP-запросов. Все компоненты системы развёрнуты в Docker-контейнерах и объединены с помощью обратного прокси-сервера Nginx. Использование контейнеризации обеспечивает изоляцию сервисов, упрощает процесс развёртывания и повышает масштабируемость системы.

Сервисы и их роли:

- Nginx — точка входа в приложение. Перенаправляет запросы /api на серверную часть, а остальные — на frontend-приложение.
- Frontend — клиентская часть приложения, реализующая пользовательский интерфейс. Взаимодействует с backend через API-запросы.
- Backend (NestJS) — серверная часть приложения, реализующая REST API и бизнес-логику. Выполняет обработку запросов, аутентификацию пользователей, валидацию данных и работу с базой данных MongoDB.
- MongoDB — база данных приложения. Хранит информацию о пользователях, картах, объектах карт и тегах.

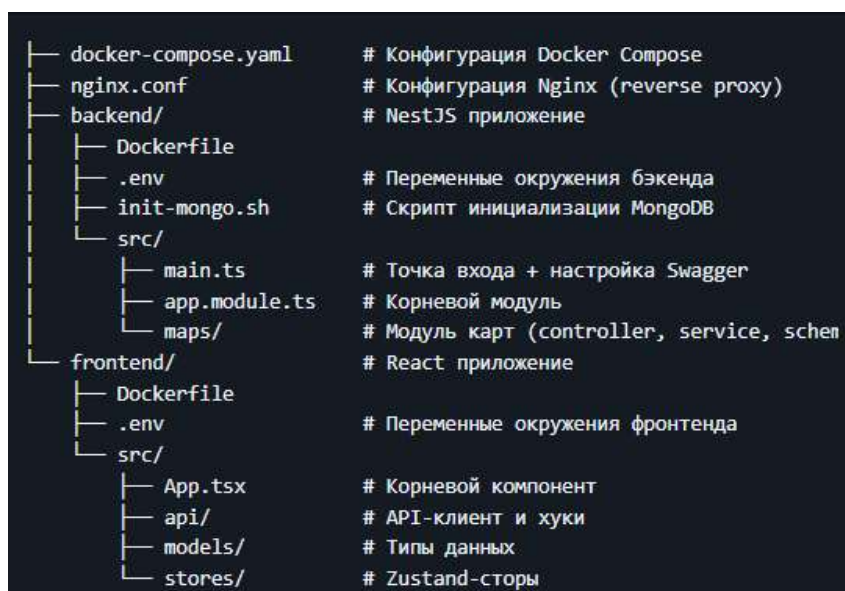


Рисунок 13 – структура проекта

Использованные технологии

Таблица 1. Использованные технологии

Слой	Технология	Обоснование выбора
Клиент	React 18 + TypeScript	Строгая типизация, компонентный подход
Картография	Leaflet + react-leaflet	Бесплатная, лёгкая, с поддержкой GeoJSON и кастомных слоёв
Управление состоянием	Zustand	Минималистичный state-менеджер, совместимый с React
Сервер	NestJS (Node.js, Express)	Модульная архитектура, встроенная поддержка TypeScript, декораторы для валидации, OpenAPI/Swagger из коробки
База данных	MongoDB + Mongoose	Документная модель отражает структуру GeoJSON и вложенных объектов, 2dsphere-индексы для пространственных запросов
Аутентификация	bcryptjs	Безопасное хранение паролей, stateless-авторизация через токены
Контейнеризация	Docker, Docker Compose	Изолированная среда, простота развёртывания на любой платформе
Обратный прокси	Nginx	Единая точка входа, проксирование API
Документация API	Swagger UI	Автоматически генерируется на основе декораторов NestJS

Снимки экрана приложения

Пользователь авторизуется или регистрируется в системе (рис. 14).

Рисунок 14 – экран входа

После входа отображается основная рабочая область (рис. 15).

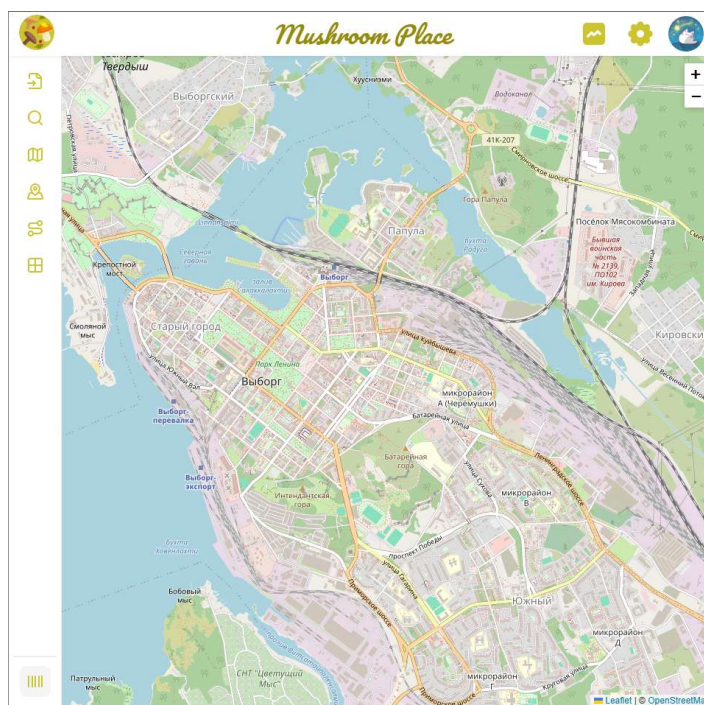


Рисунок 15 – главный экран

Пользователь может создавать карты (рис. 16), отметки (рис. 17), маршруты (рис. 18) или области (рис. 19) на карте.

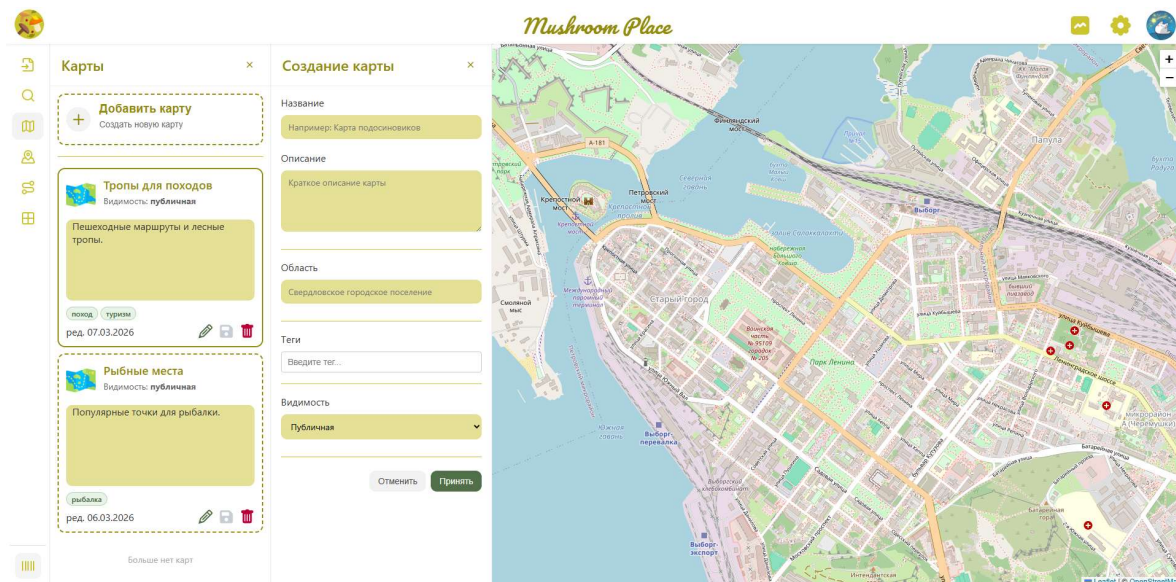


Рисунок 16 – создание карты

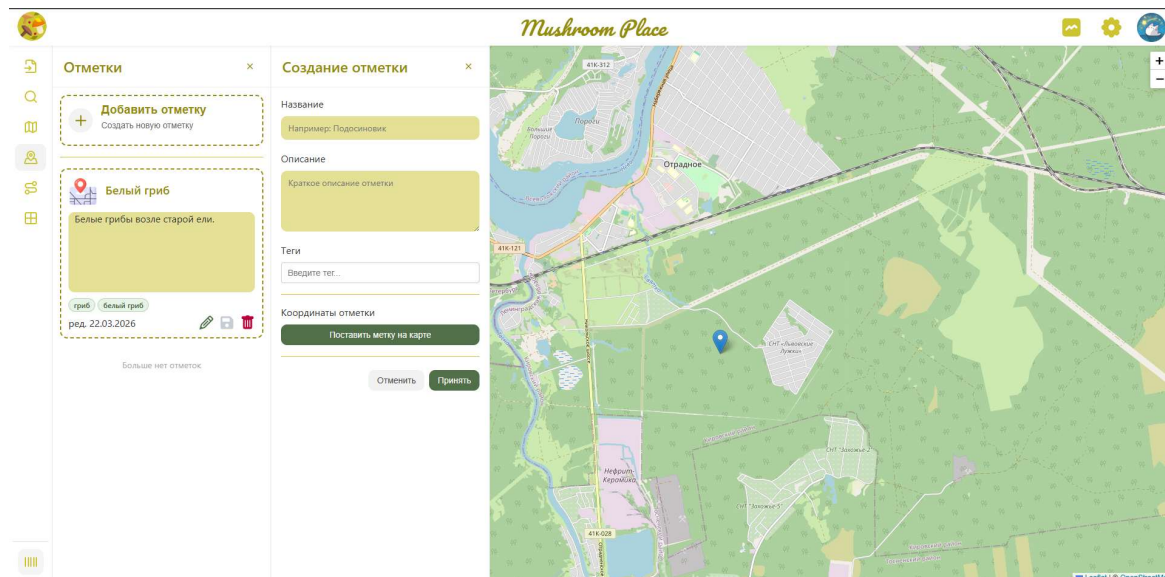


Рисунок 17 – создание отметки

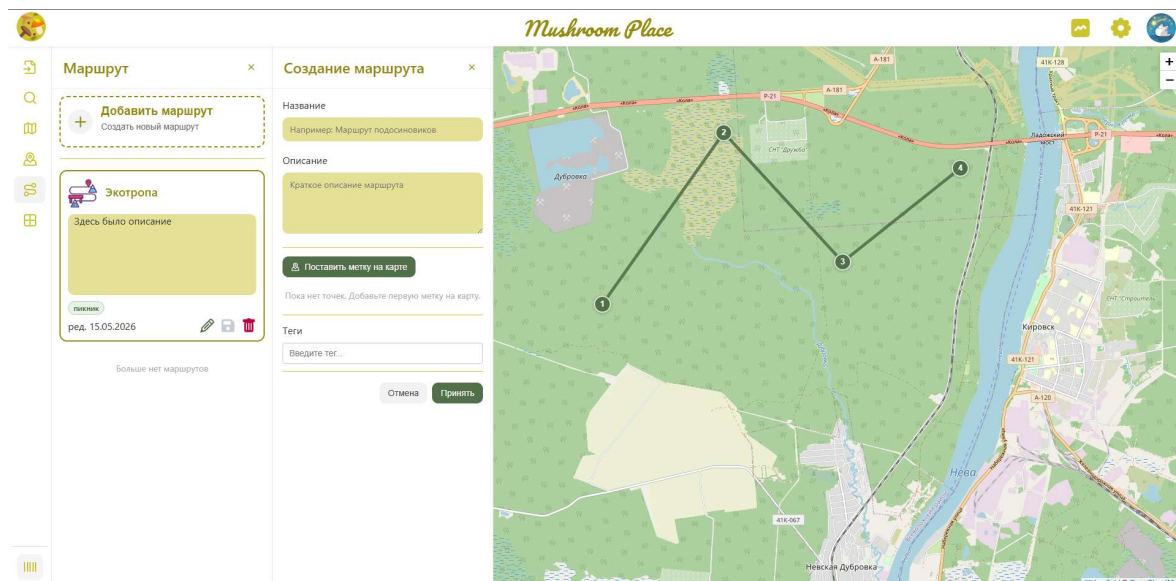


Рисунок 18 – создание маршрута

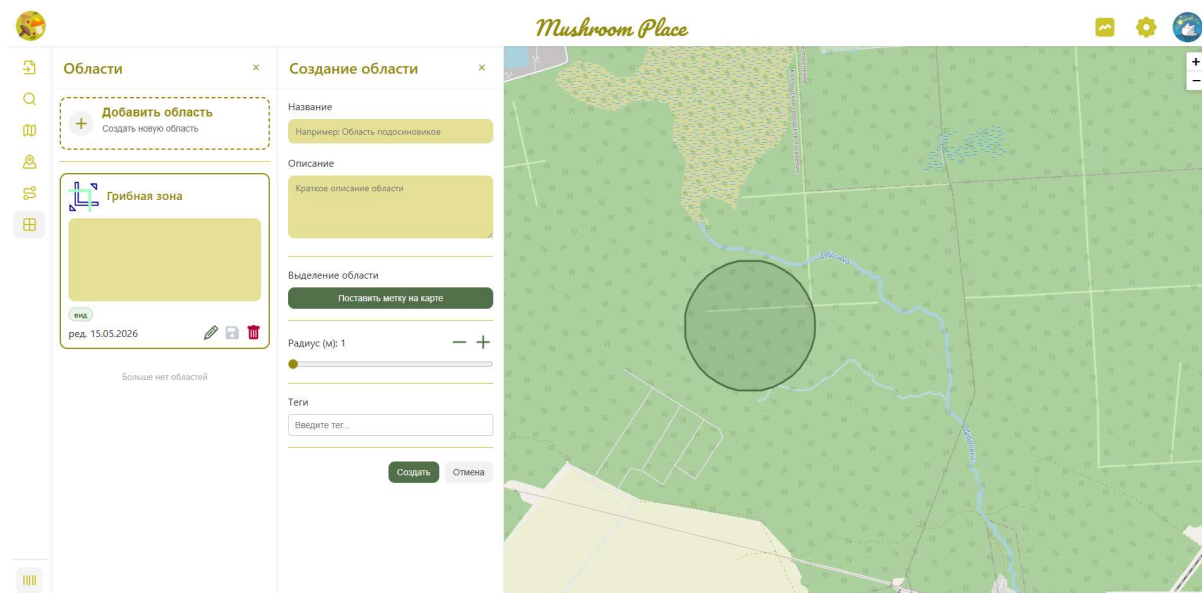


Рисунок 19 – создание области

Для упрощения поиска необходимых пользователю данных в приложении реализована функция поиска (рис. 20).

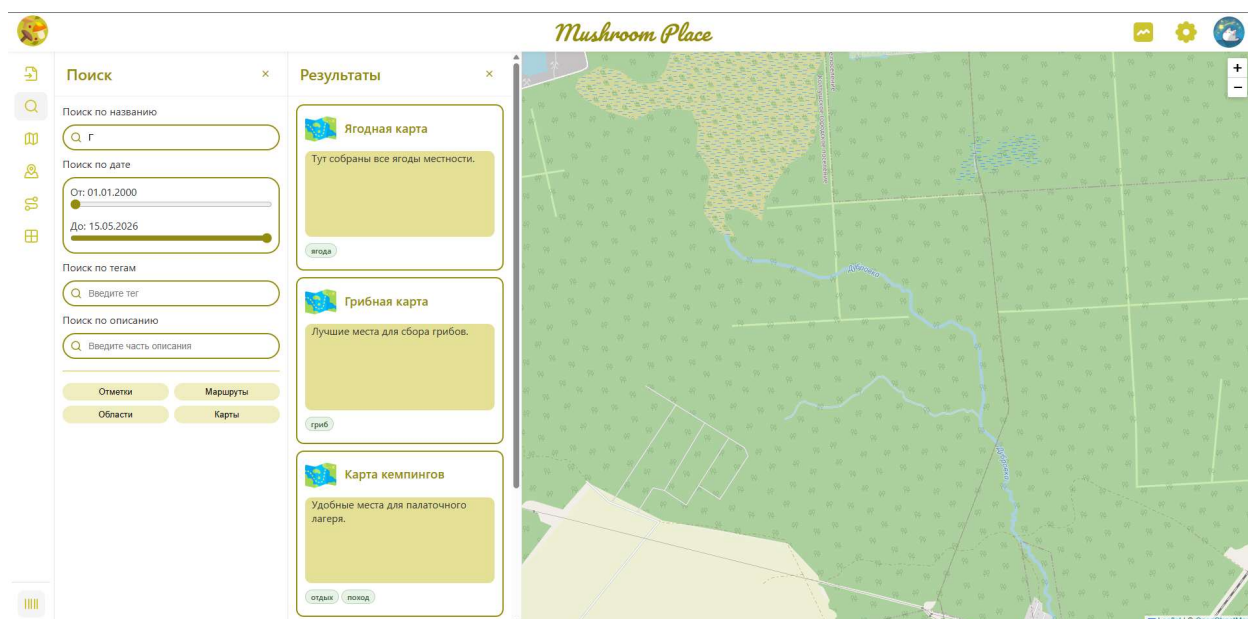


Рисунок 20 – поиск

Для удобства работы с данными в приложении реализованы функции импорта и экспорта (рис. 21).

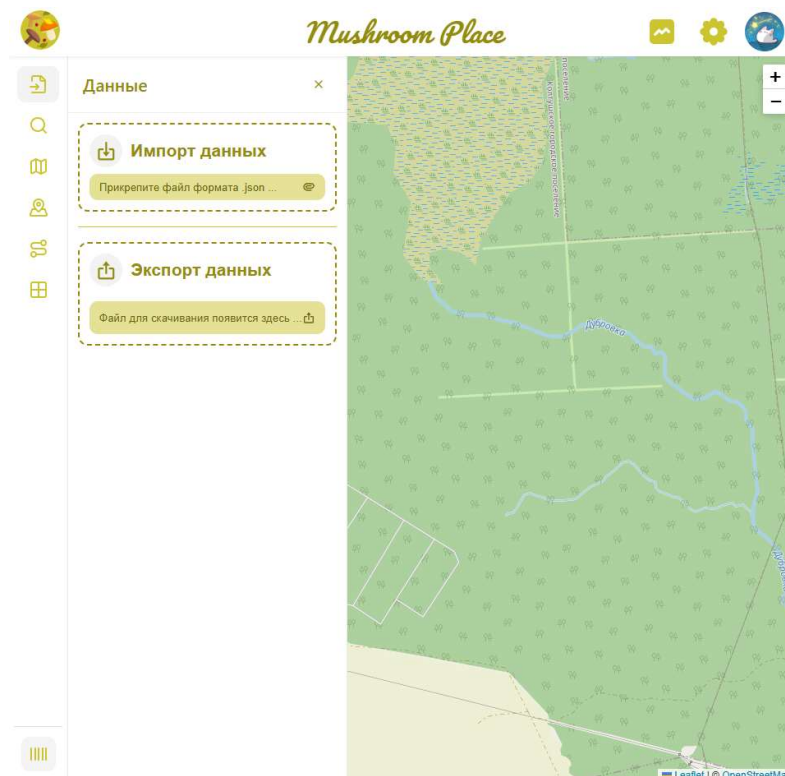


Рисунок 21 – импорт/экспорт

Для удобства анализа данных пользователем предусмотрена статистика (рис. 22).

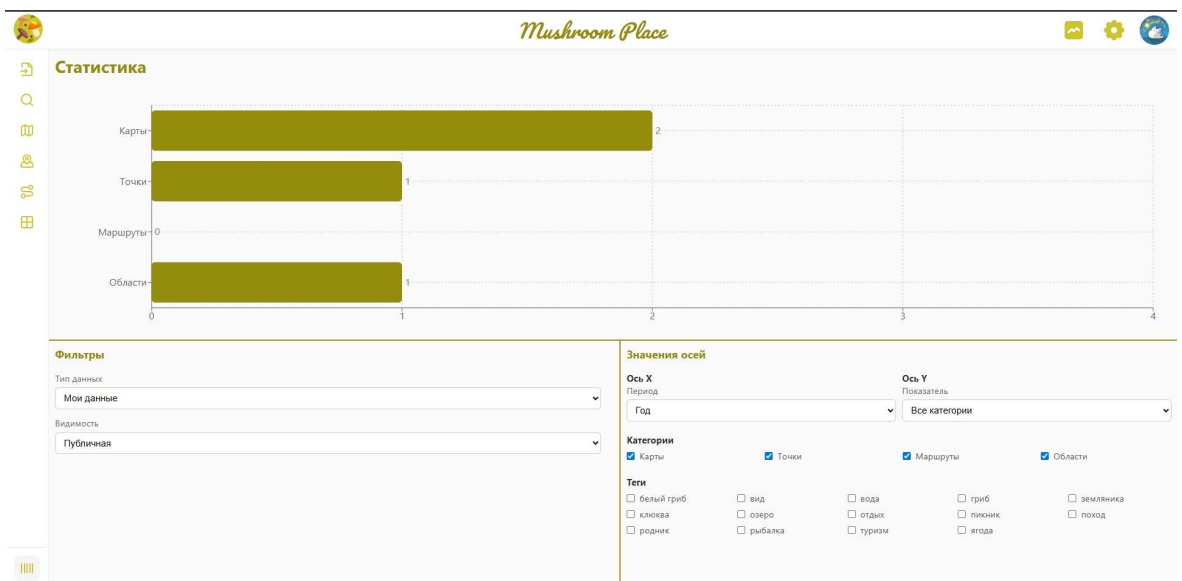


Рисунок 22 – статистика

Для удобства изменения данных пользователя предусмотрен личный кабинет (рис. 23).

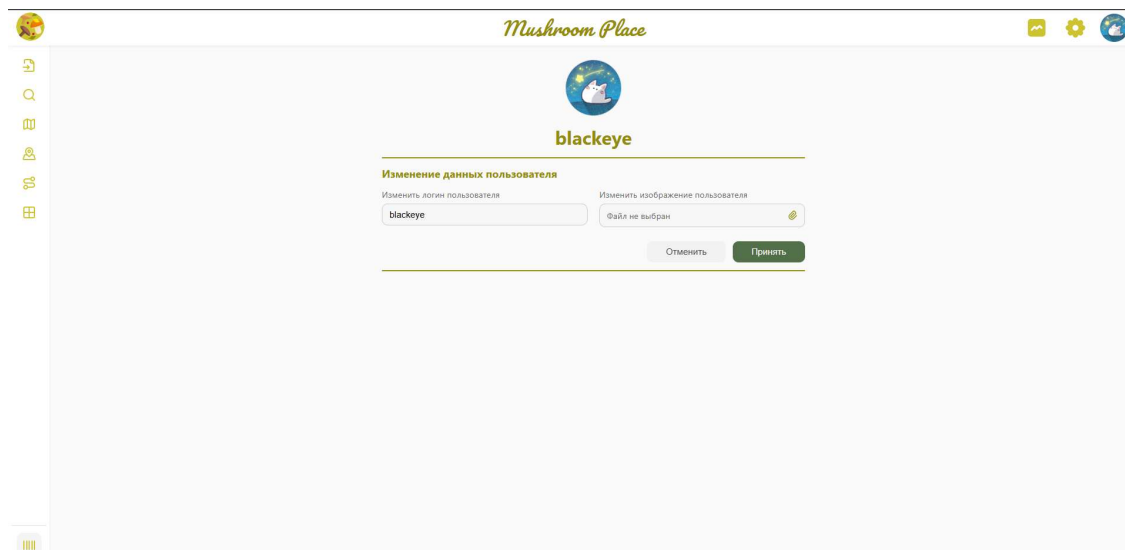


Рисунок 23 – личный кабинет

5. Вывод

Достигнутые результаты

Разработано веб-приложение «Mushrooms Place», предназначенное для работы с пользовательскими картами и геоданными. В приложении реализованы следующие основные возможности: создание, редактирование и удаление пользовательских карт; добавление и отображение объектов трёх типов: точек, маршрутов и областей; поиск по картам и объектам с фильтрацией данных; импорт и экспорт данных в формате json; отображение статистики и аналитики с построением диаграмм; аутентификация пользователей и разграничение доступа к данным; REST API с автоматически формируемой документацией Swagger. Серверная часть приложения реализована на NestJS с использованием MongoDB для хранения данных, а клиентская часть разработана на React. Для контейнеризации и развёртывания приложения использовались Docker Compose и Nginx.

Дополнительно был проведён сравнительный анализ MongoDB и PostgreSQL для хранения геоданных. Результаты показали, что MongoDB является более подходящим решением для данной предметной области благодаря удобной работе с форматом GeoJSON, меньшей избыточности данных и более эффективной обработке вложенных структур.

Недостатки и пути для улучшения полученного решения

В системе отсутствует механизм совместного доступа к картам между пользователями. В дальнейшем возможно добавление ролевой модели с разграничением прав доступа (владелец, редактор, наблюдатель).

Интерфейс приложения в основном ориентирован на настольные устройства, поэтому мобильная версия требует дополнительной адаптации элементов интерфейса и навигации.

При работе с очень большим количеством объектов на карте возможно снижение производительности клиентской кластеризации. Для повышения эффективности могут использоваться серверная кластеризация или векторные тайлы.

Кроме того, текущая версия приложения не поддерживает полноценную загрузку и хранение изображений: поле изображения хранит только строковый путь к файлу. В дальнейшем возможно добавление модуля загрузки файлов и интеграции с файловым или облачным хранилищем.

Будущее развитие решения

В дальнейшем приложение может быть расширено за счёт добавления социальных функций, таких как комментарии, рейтинги и отметки «нравится» для публичных карт и

объектов, а также формирования ленты пользовательской активности.

Перспективным направлением развития является интеграция с метеосервисами для отображения погодных условий и предоставления рекомендаций, связанных с сезонами сбора грибов.

Для повышения удобства использования может быть реализован оффлайн-режим с кэшированием карт и синхронизацией данных после восстановления подключения к сети.

Кроме того, в дальнейшем планируется полный переход на хранение геоданных в формате GeoJSON, включая использование полигонов для более точного описания областей на карте.

6. Приложения

Документация по сборке и развертыванию приложения

Требования: Docker $\geq$ 20.10, Docker Compose $\geq$ 1.29.

Шаги для локального запуска:

- Клонировать репозиторий

```
git clone https://github.com/yourusername/mushrooms-place.git
cd mushrooms-place
```

- Создайте конфигурационные файлы из шаблонов

```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env
```

- Запустите сборку и все сервисы:

```
docker-compose up --build
```

При первом запуске будет выполнена сборка образов для фронтенда и бэкенда. MongoDB инициализируется автоматически.

- После успешного старта:

Фронтенд доступен по адресу <http://localhost>

Бэкенд API проксируется туда же, эндпоинты доступны по префиксу /api

Интерактивная документация Swagger UI: <http://localhost/api/docs>

- Остановка приложения:

```
docker-compose down
```

Инструкция для пользователя

Начало работы:

1. Откройте приложение в браузере. Если у вас нет учётной записи, нажмите «Зарегистрироваться» и введите желаемые имя пользователя и пароль.
2. После регистрации войдите под своими данными.

Создание карты:

1. Нажмите «Карты» в боковом меню, затем «Добавить карту».

2. Введите название (например, «Грибные места Подмосковья»), выберите страну и субъект, задайте видимость (публичная карта видна другим, приватная – только вам).
3. При желании добавьте теги и описание.
4. Нажмите «Принять». Новая карта появится в списке и автоматически активируется.

Добавление объектов на карту:

1. Убедитесь, что нужная карта выбрана в боковой панели.
2. Чтобы поставить точку грибного места: перейдите во вкладку «Отметки» → «Добавить отметку». Кликните на карте в нужном месте, заполните название и описание, добавьте теги. Нажмите «Принять».
3. Для создания области: перейдите во вкладку «Области» → «Добавить область». Кликом укажите центр, введите радиус (в метрах) и название.
4. Для построения маршрута: перейдите во вкладку «Маршруты» → «Добавить маршрут». Последовательно кликайте на карте для установки точек маршрута (минимум две). Введите название и описание.

Импорт: перейдите во вкладку «Данные» → блок «Импорт данных». Нажмите «Прикрепите файл», выберите подготовленный .json. Проверьте, что файл корректен, и нажмите «Загрузить». После успешного импорта в списке появятся новые объекты.

Экспорт: в блоке «Экспорт данных», нажмите «Экспортировать данные». Файл будет скачан на ваше устройство.

Поиск:

1. Введите ключевые слова в строку поиска сверху. Можете уточнить область поиска, выбрав фильтр: «Все», «Карты», «Отметки», «Маршруты», «Области».
2. Система покажет список совпадений. Кликните по любому результату – откроется соответствующая карта и/или выделится объект.

Просмотр статистики:

1. Перейдите в раздел статистики.
2. Выберите значения для осей, при необходимости примените фильтры.
3. Система отобразит график с заданными параметрами или информативное сообщение о его отсутствии.

Управление профилем:

1. Перейдите в личный кабинет.
2. Измените нужные данные.
3. Нажмите кнопку «Принять» для сохранения изменений или кнопку «Отменить» для сброса аватара на аватар по умолчанию.

7. Используемая литература

1. Исходный код проекта: <https://github.com/moevm/nsql1h26-place>
2. Документация MongoDB: <https://www.mongodb.com/docs/manual/geospatial-queries/>
3. Leaflet – JavaScript library for interactive maps: <https://postgis.net/docs/>
4. NestJS – A progressive Node.js framework: <https://docs.nestjs.com/>
5. Методические указания курса «NoSQL Введение»:
https://se.moevm.info/doku.php/staff/courses:no_sql_introduction:calculating_data_model_size
6. Zustand – State management for React: <https://docs.pmnd.rs/zustand>
7. Docker Compose documentation: <https://docs.docker.com/compose/>